

COURSE

Основы программирования JS

Объекты и массивы

shesnokov@gmail.com

ОГЛАВЛЕНИЕ

План лекции *3*

Теоретическая часть *3*

Понятие объекта. **3**

Создание объектов. **4**

Использование инициализаторов объекта. 4

Использование функции конструктора. 5

Использование метода Object.create. 6

Свойства объекта. **7**

Перечисление всех свойств объекта. 9

Удаление свойств объекта. 10

Проверка существования свойства объекта. 10

Методы объектов. **11**

Ссылка на объект. **12**

Геттеры и сеттеры. **13**

Сравнение объектов. **15**

Сериализация объекта. **15**

Понятие массива. **16**

Создание массивов. **16**

Чтение и запись элементов массивов. **17**

Добавление и удаление элементов массивов. **18**

Многомерные массивы. **19**

Методы класса Array. **19**

Метод join(). 20

Метод reverse(). 20

Метод sort() 20

Метод concat() 21

Метод slice() 21

Метод splice() 21

Методы push() и pop() 22

Методы unshift() и shift(). 22

Литература и другие источники *23*

Вопросы для самопроверки *23*

ПЛАН ЛЕКЦИИ

1. Понятие объекта;
2. Свойства объекта;
3. Объект как массив;
4. Стандартные свойства и методы объектов;
5. Понятие массива;
6. Доступ к элементам массива;
7. Добавление нового элемента в массив;
8. Длина массива;
9. Методы массивов.

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

ПОНЯТИЕ ОБЪЕКТА.

JavaScript спроектирован на основе простой парадигмы. В основе концепции лежат простые объекты. ***Объект*** — это набор свойств, и каждое свойство состоит из имени и значения, ассоциированного с этим именем. Значением свойства может быть функция, которую можно назвать методом объекта. В дополнение к встроенным в браузер объектам, вы можете определить свои собственные объекты.

Объекты в языке JavaScript являются ***динамическими*** - обычно они позволяют добавлять и удалять свойства - но они могут использоваться также для имитации статических объектов и «структур», которые имеются в языках программирования со статической системой типов. Кроме того, они могут использоваться (если не учитывать, что объекты отображают строки в значения) для представления множеств строк.

Любое значение в языке JavaScript, не являющееся строкой, числом, true, false, null или undefined, является объектом. И даже строки, числа и логические значения, не являющиеся объектами, могут вести себя как неизменяемые объекты (имеются объекты-обертки String, Number и т.п.).

Объекты являются изменяемыми значениями и операции с ними ***выполняются по ссылке, а не по значению***. Если переменная x ссылается на объект, и выполняется инструкция var y = x;, в переменную y будет записана ссылка на тот же самый объект, а не его копия. Любые изменения, выполняемые в объекте с помощью переменной y, будут также отражаться на переменной x.

Свойство объекта имеет имя и значение. ***Именем свойства*** может быть любая строка, включая и пустую строку, но объект не может иметь два свойства с одинаковыми именами. ***Значением свойства*** может быть любое значение, допустимое в языке JavaScript, или (в ECMAScript 5) функция чтения или записи (или обе).

В дополнение к именам и значениям каждое свойство имеет ряд ассоциированных с ним значений, которые называют ***атрибутами свойства***:

- атрибут ***writable*** определяет доступность значения свойства для записи.
- атрибут ***enumerable*** определяет доступность имени свойства для перечисления в цикле for/in.
- атрибут ***configurable*** определяет возможность настройки, т.е. удаления свойства и изменения его атрибутов.

До появления стандарта ECMAScript 5 все свойства в объектах, создаваемые программой, доступны для записи, перечисления и настройки. В ECMAScript 5 предусматривается возможность настройки атрибутов ваших свойств.

В дополнение к свойствам каждый объект имеет три атрибута объекта:

- атрибут ***prototype*** содержит ссылку на другой объект, от которого наследуются свойства.
- атрибут ***class*** содержит строку с именем класса объекта и определяет тип объекта.
- флаг ***extensible*** (в ECMAScript 5) указывает на возможность добавления новых свойств в объект.

Объект базового языка - это объект или класс объектов, определяемый спецификацией ECMAScript. Массивы, функции, даты и регулярные выражения (например) являются объектами базового языка.

Объект среды выполнения - это объект, определяемый средой выполнения (такой как веб-браузер), куда встроен интерпретатор JavaScript. Объекты HTMLElement, представляющие структуру веб-страницы в клиентском JavaScript, являются объектами среды выполнения. Объекты среды выполнения могут также быть объектами базового языка, например, когда среда выполнения определяет методы, которые являются обычными объектами Function базового языка JavaScript.

Пользовательский объект - любой объект, созданный в результате выполнения программного кода JavaScript.

Собственное свойство - это свойство, определяемое непосредственно в данном объекте.

Унаследованное свойство - это свойство, определяемое прототипом объекта.

Создание объектов.

JavaScript содержит набор встроенных объектов. Также вы можете создавать свои объекты. Начиная с JavaScript 1.2, вы можете создавать объект с помощью инициализатора объекта. Другой способ — создать функцию-конструктор и сделать экземпляр объекта с помощью этой функции и ***оператора new***.

Использование инициализаторов объекта.

Помимо создания объектов с помощью функции-конструктора вы можете создавать объекты и другим, особым способом. Фактически, вы можете записать объект синтаксически, и он будет создан интерпретатором автоматически во время выполнения. Эта синтаксическая схема приведена ниже:

```
var obj = {
    property_1:   value_1, // property_# may be an identifier...
    2:            value_2, // or a number...
    // ...,
    "property n": value_n
}; // or a string
```

здесь ***obj*** — это имя нового объекта, каждое ***property_i*** — это идентификатор (имя, число или строковый литерал), и каждый ***value_i*** — это значения, назначенные ***property_i***.

Имя obj и ссылка объекта на него необязательна; если далее вам не надо будет ссылаться на данный объект, то вам не обязательно назначать объект переменной (обратите внимание, что вам потребуется обернуть литерал объекта в скобки, если объект находится в месте, где ожидается инструкция, чтобы интерпретатор не перепутал его с блоком.)

Если объект создан при помощи инициализатора объектов на высшем уровне скрипта, то JavaScript интерпретирует объект каждый раз, когда анализирует выражение, содержащее объект, записанный как литерал. Плюс, если пользоваться функцией инициализатором, то он будет создаваться каждый раз, когда функция вызывается.

Следующая инструкция создает объект и назначает его переменной x, когда выражение cond истинно.

```
if (cond) var x = {hi: "there"};
```

Следующий пример создает объект myHonda с тремя свойствами. Заметьте, что свойство engine — это также объект со своими собственными свойствами.

```
var myHonda = {
        color: "red",
        wheels: 4,
        engine: {
            cylinders: 4,
            size: 2.2
        }
};
```

Вы также можете использовать инициализатор объекта для создания массивов.

Использование функции конструктора.

Другой способ создать объект в два шага описан ниже:

- Определите тип объекта, написав ***функцию-конструктор***. Название такой функции, как правило, начинается с заглавной буквы.
- Создайте экземпляр объекта с помощью ключевого слова ***new***.

Чтобы определить тип объекта создайте функцию, которая определяет тип объекта, его имя, свойства и методы. Например, предположим, что вы хотите создать тип объекта для описания машин. Вы хотите, чтобы объект этого типа назывался car, и вы хотите, чтобы у него были свойства make, model, и year. Чтобы сделать это, напишите следующую функцию:

```
function Car(make, model, year) {
        this.make = make;
        this.model = model;
        this.year = year;
}
```

Заметьте, что используется this чтобы присвоить значения (переданные как аргументы функции) свойствам объекта. Теперь вы можете создать объект, называемый mycar, следующим образом:

```
var mycar = new Car("Eagle", "Talon TSi", 1993);
```

Эта инструкция создает объект типа Car с ссылкой mycar и присваивает определенные значения его свойствам. Значением mycar.make станет строка "Eagle", mycar.year — это целое число 1993, и так далее.

Вы можете создать столько объектов car, сколько нужно, просто вызывая new. Например:

```
var kenscar = new Car("Nissan", "300ZX", 1992);
var vpgscar = new Car("Mazda", "Miata", 1990);
```

Объект может иметь свойство, которое будет другим объектом. Например, далее определяется объект типа Person следующим образом:

```
function Person(name, age, country) {
        this.name = name;
        this.age = age;
        this.country = country;
}
```

и затем создать два новых экземпляра объектов person как показано далее:

```
var rand = new Person("Rand McKinnon", 33, "M");
var ken = new Person("Ken Jones", 39, "M");
```

Затем, вы можете переписать определение car и включить в него свойство owner, которому назначить объект person следующим образом:

```
function Car(make, model, year, owner) {
        this.make = make;
        this.model = model;
        this.year = year;
        this.owner = owner;
}
```

Затем, чтобы создать экземпляры новых объектов, выполните следующие инструкции:

```
var car1 = new Car("Eagle", "Talon TSi", 1993, rand);
var car2 = new Car("Nissan", "300ZX", 1992, ken);
```

Заметьте, что вместо того, чтобы передавать строку, литерал или целое число при создании новых объектов, в выражениях выше передаются объекты rand и ken как аргумент функции. Теперь, если вам нужно узнать имя владельца car2, это можно сделать следующим образом:

```
car2.owner.name
```

Заметьте, что в любое время вы можете добавить новое свойство ранее созданному объекту. Например, выражение

```
car1.color = "black";
```

добавляет свойство color к car1, и устанавливая его значение равным "black." Как бы там ни было, это не влияет на любые другие объекты. Чтобы добавить новое свойство всем объектам одного типа, вы должны добавить свойство в определение типа объекта car.

Использование метода Object.create.

Объекты также можно создавать с помощью ***метода Object.create***. Этот метод очень удобен, так как позволяет вам указывать объект прототип для нового вашего объекта без определения функции конструктора.

```
// список свойств и методов для Animal
var Animal = {
        type: 'Invertebrates', // Значение type по умолчанию
```

```
        displayType: function() {  // Метод отображающий тип объекта Animal
            console.log(this.type);
        }
};

// Создаем объект Animal
var animal1 = Object.create(Animal);
animal1.displayType(); // Выведет:Invertebrates

// Создаем объект Animal и присваиваем ему type = Fishes
var fish = Object.create(Animal);
fish.type = 'Fishes';
fish.displayType(); // Выведет:Fishes
```

СВОЙСТВА ОБЪЕКТА.

В JavaScript объект имеет свойства, ассоциированные с ним. ***Свойство объекта*** можно понимать, как переменную, закрепленную за объектом. Свойства объекта в сущности являются теми же самыми переменными JavaScript, за тем исключением, что они закреплены за объектом. Свойства объекта определяют его характеристики. Получить доступ к свойству объекта можно так:

objectName.propertyName

Как и все переменные JavaScript, имя объекта (которое тоже может быть переменной) и имя свойства являются чувствительными к регистру. Вы определяете свойство, просто указывая его значение. Например, давайте создадим объект myCar и определим его свойства make, model, и year следующим образом:

```
var myCar = new Object();
myCar.make = "Ford";
myCar.model = "Mustang";
myCar.year = 1969;
```

Доступ к свойствам объектов JavaScript можно получить через ***квадратные скобки***. Объекты являются ассоциативными массивами, так как каждое свойство ассоциировано с именем, через которое можно получить доступ к нему. Например, доступ к свойствам объекта myCar можно получить следующим образом:

```
myCar["make"] = "Ford";
myCar["model"] = "Mustang";
myCar["year"] = 1969;
```

Имена свойств объекта могут быть строками JavaScript, или тем, что может быть сконвертировано в строку, включая пустую строку. Как бы то ни было, доступ к любому имени свойства, которое содержит невалидный JavaScript идентификатор (например, имя свойства содержит в себе пробел и тире или

начинается с цифры), может быть получен с использованием квадратных скобок. Этот способ также полезен в случае если имена свойств определяются динамически в процессе выполнения скрипта. Примеры далее:

```
var myObj = new Object(),
        str = "myString",
        rand = Math.random(),
        obj = new Object();

myObj.type = "Dot syntax";
myObj["date created"] = "String with space";
myObj[str] = "String value";
myObj[rand] = "Random Number";
myObj[obj] = "Object";
myObj[""] = "Even an empty string";

console.log(myObj);
```

Вы также можете получить доступ к свойству, воспользовавшись строковой переменной в которой хранится имя свойства:

```
var propertyName = "make";
myCar[propertyName] = "Ford";
propertyName = "model";
myCar[propertyName] = "Mustang";
```

Вы можете пользоваться квадратными скобками в конструкции for...in чтобы выполнить итерацию всех свойств объекта, для которых она разрешена. Чтобы показать, как это работает, следующая функция показывает все свойства объекта, когда вы передаете в нее сам объект и его имя как аргументы функции:

```
function showProps(obj, objName) {
        var result = "";
        for (var i in obj) {
            if (obj.hasOwnProperty(i)) {
                result += objName + "." + i + " = " + obj[i] + "\n";
            }
        }
        return result;
}
```

Так что если вызвать эту функцию вот так showProps(myCar, "myCar"), то получим результат:

```
myCar.make = Ford
myCar.model = Mustang
myCar.year = 1969
```

Перечисление всех свойств объекта.

Начиная с ECMAScript 5, есть три способа перечислить все свойства объекта (получить их список):

- циклы for...in - этот метод перебирает все перечисляемые свойства объекта и его цепочку прототипов;
- Object.keys(o) - этот метод возвращает массив со всеми собственными (те, что в цепочке прототипов, не войдут в массив) именами перечисляемых свойств объекта o.
- Object.getOwnPropertyNames(o) - этот метод возвращает массив, содержащий все имена своих свойств (перечисляемых и неперечисляемых) объекта o.

В ECMAScript 5 нет родного способа перечислить все свойства объекта. Однако, это можно сделать с помощью следующей функции:

```
function listAllProperties(o) {
        var objectToInspect;
        var result = [];
        for(objectToInspect = o; objectToInspect !== null; objectToInspect = Object.getPrototypeOf(objectToInspect)) {
                result = result.concat(Object.getOwnPropertyNames(objectToInspect));
        }
        return result;
}
```

Это может быть полезно для обнаружения скрытых (hidden) свойств (свойства в цепочке прототипа, которые недоступны через объект, в случае, если другое свойство имеет такое же имя в предыдущем звене из цепочки прототипа). Перечислить доступные свойства можно, если удалить дубликаты из массива.

Вместо проверки наличия отдельных свойств иногда бывает необходимо обойти все имеющиеся свойства или получить список всех свойств объекта. Обычно для этого используется цикл for/in, однако стандарт ECMAScript предоставляет две удобные альтернативы.

Инструкция цикла for/in выполняет тело цикла для каждого перечислимого свойства (собственного или унаследованного) указанного объекта, присваивая имя свойства переменной цикла. Встроенные методы, наследуемые объектами, являются неперечислимыми, а свойства, добавляемые в объекты вашей программой, являются перечислимыми (если только не использовались функции, описываемые ниже, позволяющие сделать свойства неперечисляемым). Например,

```
// Простой объект с тремя перечислимыми свойствами
var user = { login : 'kot86', name : 'Alexandr', age : 26 };
user.propertyIsEnumerable('toString');          // false, toString - встроенный метод
for (n in user)
        console.log(n);
```

Некоторые библиотеки добавляют новые методы (или другие свойства) в объект Object.prototype, чтобы они могли быть унаследованы и быть доступны всем объектам. Однако до появления стандарта

ECMAScript отсутствовала возможность сделать эти дополнительные методы неперечислимыми, поэтому они оказывались доступными для перечисления в циклах for/in. Чтобы решить эту проблему, может потребоваться фильтровать свойства, возвращаемые циклом for/in. Ниже приводятся два примера реализации такой фильтрации:

```
for (n in user) {
        if (!user.hasOwnProperty(n)) continue;
        console.log(n);
}
for (n in user) {
        if (typeof user[n] === 'function') continue;
        console.log(n);
}
```

В дополнение к циклу for/in стандарт ECMAScript определяет две функции, перечисляющие имена свойств. Первая из них, Object.keys(), возвращает массив имен собственных перечислимых свойств объекта.

Вторая функция ECMAScript, выполняющая перечисление свойств - ***Object.getOwnPropertyNames()***. Она действует подобно функции Object.keys(), но возвращает имена всех собственных свойств указанного объекта, а не только перечислимые. В реализациях ECMAScript 3 отсутствует возможность реализовать подобные функции, потому что ECMAScript 3 не предусматривает возможность получения неперечисляемых свойств объекта.

Удаление свойств объекта.

Вы можете удалить свойство используя оператор delete. Следующий код показывает, как удалить свойство.

```
//Creates a new object, myobj, with two properties, a and b.
var myobj = new Object;
myobj.a = 5;
myobj.b = 12;
//Removes the a property, leaving myobj with only the b property.
delete myobj.a;
```

Вы также можете воспользоваться delete чтобы удалить глобальную переменную, если ключевое слово var не было использовано при ее объявлении:

```
g = 17;
delete g;
```

Проверка существования свойства объекта.

Объекты в языке JavaScript можно рассматривать как множества свойств, и нередко бывает полезно иметь возможность проверить принадлежность к множеству - проверить наличие в объекте свойства с данным именем. Выполнить такую проверку можно с помощью ***оператора in***, с помощью ***методов hasOwnProperty() и propertyIsEnumerable()*** или просто обратившись к свойству.

Оператор in требует, чтобы в левом операнде ему было передано имя свойства (в виде строки) и объект в правом операнде. Он возвращает true, если объект имеет собственное или унаследованное свойство с этим именем:

```
var obj = { x:1 }
"x" in obj;          // true: obj имеет собственное свойство "x"
"y" in obj;          // false: obj не имеет свойства "y"
"toString" in obj;   // true: obj наследует свойство toString
```

Метод hasOwnProperty() объекта проверяет, имеет ли объект собственное свойство с указанным именем. Для наследуемых свойств он возвращает false:

```
var obj = { x:1 }
obj.hasOwnProperty('x');          // true: obj имеет собственное свойство "x"
obj.hasOwnProperty('y');          // false: obj не имеет свойства "y"
obj.hasOwnProperty('toString');   // false: toString - наследуемое свойство
```

Метод propertyIsEnumerable() накладывает дополнительные ограничения по сравнению с hasOwnProperty(). Он возвращает true, только если указанное свойство является собственным свойством, атрибут enumerable которого имеет значение true. Свойства встроенных объектов не являются перечислимыми. Свойства, созданные обычной программой на языке JavaScript, являются ***перечислимыми***, если не был использован один из методов ECMAScript, представленных ниже, которые делают свойства ***неперечислимыми***.

Часто вместо оператора in достаточно использовать простое выражение обращения к свойству и использовать оператор !== для проверки на неравенство значению undefined:

```
var obj = { x:1 }
obj.x !== undefined;          // true: obj имеет свойство "x"
obj.y !== undefined;          // false: obj не имеет свойства "y"
obj.toString !== undefined;   // true: obj наследует свойство toString
```

Однако оператор in отличает ситуации, которые неотличимы при использовании представленного выше приема на основе обращения к свойству. Оператор in отличает отсутствие свойства от свойства, имеющего значение undefined.

Методы объектов.

Метод — это функция, ассоциированная с объектом или, проще говоря, метод — это свойство объекта, являющееся функцией. Методы определяются так же, как и обычные функции, за тем исключением, что они присваиваются свойству объекта. Например, вот так:

```
objectName.methodname = function_name;
var myObj = {
        myMethod: function(params) {
            // ...do something
        }
};
```

где ***objectName*** - это существующий объект, ***methodname*** - это имя, которое вы присваиваете методу, и ***function_name*** — это имя самой функции.

Затем вы можете вызвать метод в контексте объекта следующим образом:

object.methodname(params);

Вы можете определять методы для типа объекта, включая определение метода в функцию конструктора объекта. Например, вы можете определить функцию, которая форматирует и отображает свойства до этого определенных объектов car. Например,

```
function displayCar() {
        var result = "A Beautiful " + this.year + " " + this.make + " " + this.model;
        pretty_print(result);
}
```

где ***pretty_print*** — это функция отображения горизонтальной линии и строки. Заметьте, что использование this позволяет ссылаться на объект, к которому принадлежит метод.

Вы можете сделать эту функцию методом car, добавив инструкцию

```
this.displayCar = displayCar;
```

к определению объекта. Таким образом, полное определение car примет следующий вид:

```
function Car(make, model, year, owner) {
        this.make = make;
        this.model = model;
        this.year = year;
        this.owner = owner;
        this.displayCar = displayCar;
}
```

Теперь вы можете вызвать метод displayCar для каждого из объектов как показано ниже:

```
car1.displayCar();
car2.displayCar();
```

Ссылка на объект.

В JavaScript есть специальное ключевое слово this, которое вы можете использовать внутри метода, чтобы ссылаться на текущий объект. Предположим, у вас есть функция validate, которая сверяет свойство value, переданного ей объекта с некоторыми верхним и нижним значениями:

```
function validate(obj, lowval, hival) {
        if ((obj.value < lowval) || (obj.value > hival))
                alert("Invalid Value!");
}
```

Вы можете вызвать эту функцию validate в каждом элементе формы, в обработчике ***события onchange***. Используйте this для доступа к этому элементу, как это сделано ниже:

```
<input type="text" name="age" size="3"
        onChange="validate(this, 18, 99)">
```

В общем случае, this ссылается на объект, вызвавший метод.

Через this можно обратиться и к родительской форме элемента, воспользовавшись свойством form. В следующем примере форма myForm содержит элемент ввода Text и кнопку button1. Когда пользователь нажимает кнопку, значению объекта Text назначается имя формы. Обработчик событий кнопки onclick пользуется this.form чтобы сослаться на текущую форму, myForm.

```
<form name="myForm">
	<p><label>Form name:<input type="text" name="text1" value="Beluga"></label></p>
	<p><input name="button1" type="button" value="Show Form Name"
		onclick="this.form.text1.value = this.form.name">
	</p>
</form>
```

Геттеры и сеттеры.

Геттер (от англ. get - получить) - это метод, который получает значение определенного свойства. ***Сеттер (от англ. set — присвоить)*** - это метод, который присваивает значение определенному свойству объекта. Вы можете определить геттеры и сеттеры для любых из встроенных или определенных вами объектов, которые поддерживают добавление новых свойств. Синтаксис определения геттеров и сеттеров использует литеральный синтаксис объектов.

Ниже проиллюстрировано, как могут работать геттеры и сеттеры в объекте, определенном пользователем:

```
var o = {
	a: 7,
	get b() {
		return this.a + 1;
	},
	set c(x) {
		this.a = x / 2;
	}
};

console.log(o.a); // 7
console.log(o.b); // 8
o.c = 50;
console.log(o.a); // 25
```

Объект o получит следующие свойства:

- o.a - число;
- o.b - геттер, который возвращает o.a плюс 1;
- o.c - сеттер, который присваивает значение o.a половине значения которое передано в o.c.

Следует особо отметить, что имена функций, указанные в литеральной форме "[gs]et propertyName() { }" не будут в действительности являться именами геттера и сеттера. Чтобы задать в качестве геттера и

сеттера функции с явно определенными именами, используйте метод Object.defineProperty (или его устаревший аналог Object.prototype.__defineGetter__).

В коде ниже показано, как с помощью геттера и сеттера можно расширить прототип объекта Date и добавить ему свойство year, которое будет работать у всех экземпляров класса Date. Этот код использует существующие методы класса Date - getFullYear и setFullYear для работы геттера и сеттера.

Определение геттера и сеттера для свойства year:

```
var d = Date.prototype;
Object.defineProperty(d, 'year', {
        get: function() { return this.getFullYear(); },
        set: function(y) { this.setFullYear(y); }
});
```

Использование свойства year заданного геттером и сеттером:

```
var now = new Date();
console.log(now.year); // 2000
now.year = 2001; // 987617605170
console.log(now);
// Wed Apr 18 11:13:25 GMT-0700 (Pacific Daylight Time) 2001
```

Стоит отметить, что геттеры и сеттеры могут быть либо:

- определены при использовании инициализаторов объекта;
- добавлены существующему объекту в любой момент, при использовании методов добавления геттеров и сеттеров.

Когда определение геттера и сеттера использует инициализаторы объекта, все что вам нужно, это дополнить геттер префиксом get, а сеттер префиксом set. При этом, метод геттера не должен ожидать каких-либо параметров, в то время как метод сеттера принимает один единственный параметр (новое значение для присвоения свойству). Например,

```
var o = {
        a: 7,
        get b() { return this.a + 1; },
        set c(x) { this.a = x / 2; }
};
```

Геттеры и сеттеры, могут быть добавлены существующему объекту в любой момент, при помощи метода Object.defineProperties. Первый параметр этого метода - объект, которому вы хотите присвоить геттер и сеттер. Второй параметр - это объект, имена свойств которого будут соответствовать именам создаваемых свойств, а значения – объекты, определяющие геттер и сеттер создаваемых свойств. В следующем примере создаются в точности такие же геттер и сеттер, как и в примере выше:

```
var o = { a: 0 };
Object.defineProperties(o, {
        'b': { get: function() { return this.a + 1; } },
        'c': { set: function(x) { this.a = x / 2; } }
```

```
});
o.c = 10; // Запускает сеттер, который присваивает 10 / 2 (5) свойству 'a'
console.log(o.b); // Запускает геттер, который возвращает a + 1 (то есть 6)
```

То, какую из двух форм использовать для определения свойств, зависит от вашего стиля программирования и стоящей перед вами задачи. Если вы уже используете инициализатор объекта для определения прототипа, то, скорее всего, в большинстве случаев, вы воспользуетесь первой формой. Она более компактна и естественна. Однако, не редко, вторая форма является единственно возможной, в случаях, когда вы работаете с существующим объектом без доступа к его определению. Вторая форма наилучшим образом отражает динамическую природу JavaScript — но может сделать код сложным для чтения и понимания.

Сравнение объектов.

В JavaScript объекты имеют ссылочный тип. Два отдельных объекта никогда не будут равными, даже если они имеют равный набор свойств. Только сравнение двух ссылок на один и тот же объект вернет true.

```
// Две переменных ссылаются на два объекта с одинаковыми свойствами
var fruit = {name: 'apple'};
var fruitbear = {name: 'apple'};

fruit == fruitbear; // вернет false
fruit === fruitbear; // вернет false

// Две переменные ссылаются на один общий объект
var fruit = {name: 'apple'};
var fruitbear = fruit;  // присвоим переменной fruitbear ссылку на объект fruit

// теперь fruitbear и fruit ссылаются на один и тот же объект
fruit == fruitbear; // return true
fruit === fruitbear; // return true

fruit.name = 'grape';
console.log(fruitbear);    // вернет { name: "grape" } вместо { name: "apple" }
```

Сериализация объекта.

Сериализация объектов - это процесс преобразования объектов в строковую форму представления, которая позднее может использоваться для их восстановления. Для сериализации и восстановления объектов JavaScript стандартом ECMAScript предоставляются встроенные функции JSON.stringify() и JSON.parse(). Эти функции используют формат обмена данными JSON. Название JSON происходит от «JavaScript Object Notation» (форма записи объектов JavaScript), а синтаксис этой формы записи напоминает синтаксис литералов объектов и массивов в языке JavaScript:

```
var obj = {x:1, y:{z:[false,null,""]}};               // Определить испытательный объект
```

```
var s = JSON.stringify(obj);          // s == '{"x":1,"y":{"z":[false,null,""]}}'
var p = JSON.parse(s);                // p - копия объекта obj
```

Синтаксис формата JSON является лишь подмножеством синтаксиса языка JavaScript и не может использоваться для представления всех возможных значений, допустимых в JavaScript. Поддерживаются и могут быть сериализованы и восстановлены: объекты, массивы, строки, конечные числовые значения, true, false и null. Значения NaN, Infinity и -Infinity сериализуются в значение null. Объекты Date сериализуются в строки с датами в формате ISO, но JSON.parse() оставляет их в строковом представлении и не восстанавливает первоначальные объекты Date.

Объекты Function, RegExp и Error и значение undefined не могут быть сериализованы или восстановлены. Функция JSON.stringify() сериализует только перечислимые собственные свойства объекта. Если значение свойства не может быть сериализовано, это свойство просто исключается из строкового представления. Обе функции, JSON.stringify() и JSON.parse(), принимают необязательный второй аргумент, который можно использовать для настройки процесса сериализации и/или восстановления, например, посредством определения списка свойств, подлежащих сериализации, или функции преобразования значений во время сериализации.

Понятие массива.

Массив - это упорядоченная коллекция значений. Значения в массиве называются элементами, и каждый элемент характеризуется числовой позицией в массиве, которая называется индексом. Массивы в языке javascript являются нетипизированными: элементы массива могут иметь любой тип, причем разные элементы одного и того же массива могут иметь разные типы. Элементы массива могут даже быть объектами или другими массивами, что позволяет создавать сложные структуры данных, такие как массивы объектов и массивы массивов.

Отсчет индексов массивов в языке javascript начинается с нуля и для них используются 32-битные целые числа - первый элемент массива имеет индекс 0. Массивы в javascript являются динамическими: они могут увеличиваться и уменьшаться в размерах по мере необходимости; нет необходимости объявлять фиксированные размеры массивов при их создании или повторно распределять память при изменении их размеров.

Массивы в языке javascript - это специализированная форма объектов, а индексы массивов означают чуть больше, чем просто имена свойств, которые по совпадению являются целыми числами. Доступ к элементам массива.

Создание массивов.

Легче всего создать массив с помощью литерала, который представляет собой простой список разделенных запятыми элементов массива в квадратных скобках. Значения в литерале массива не обязательно должны быть константами - это могут быть любые выражения, в том числе и литералы объектов:

```
var empty = [];                       // Пустой массив
var numbers = [2, 3, 5, 7, 11];       // Массив с пятью числовыми элементами
var misc = [ 1.1, true, "a", ];       // 3 элемента разных типов + завершающая запятая

var base = 1024;
var table = [base, base+1, base+2, base+3];           // Массив с переменными
```

```
var arrObj = [[1,{x:1, y:2}], [2, {x:3, y:4}]];     // 2 массива внутри, содержащие объекты
```

Синтаксис литералов массивов позволяет вставлять необязательную завершающую запятую, т.е. литерал [,,] соответствует массиву с двумя элементами, а не с тремя.

Другой способ создания массива состоит в вызове конструктора Array(). Вызвать конструктор можно тремя разными способами:

1) Вызвать конструктор без аргументов:

```
var arr = new Array();
```

В этом случае будет создан пустой массив, эквивалентный литералу [].

2) Вызвать конструктор с единственным числовым аргументом, определяющим длину массива:

```
var arr = new Array(10);
```

В этом случае будет создан пустой массив указанной длины. Такая форма вызова конструктора Array() может использоваться для предварительного распределения памяти под массив, если заранее известно количество его элементов. Обратите внимание, что при этом в массиве не сохраняется никаких значений.

3) Явно указать в вызове конструктора значения первых двух или более элементов массива или один нечисловой элемент:

```
var arr = new Array(5, 4, 3, 2, 1, "тест");
```

В этом случае аргументы конструктора становятся значениями элементов нового массива. Использование литералов массивов практически всегда проще, чем подобное применение конструктора Array().

Чтение и запись элементов массивов.

Доступ к элементам массива осуществляется с помощью оператора []. Слева от скобок должна присутствовать ссылка на массив. Внутри скобок должно находиться произвольное выражение, возвращающее неотрицательное целое значение. Этот синтаксис пригоден как для чтения, так и для записи значения элемента массива. Следовательно, допустимы все приведенные далее JavaScript-инструкции:

```
// Создать массив с одним элементом
var arr = ["world"];

// Прочитать элемент 0
var value = arr[0];

// Записать значение в элемент 1
arr[1] = 3.14;

// Записать значение в элемент 2
i = 2; arr[i] = 3;

// Записать значение в элемент 3
arr[i + 1] = 'привет';
```

```
// Прочитать элементы 0 и 2, записать значение в элемент 3
arr[arr[i]] = arr[0];
```

Массивы являются специализированной разновидностью объектов. Квадратные скобки, используемые для доступа к элементам массива, действуют точно так же, как квадратные скобки, используемые для доступа к свойствам объекта. Интерпретатор JavaScript преобразует указанные в скобках числовые индексы в строки - индекс 1 превращается в строку "1" - а затем использует строки как имена свойств.

В преобразовании числовых индексов в строки нет ничего особенного: то же самое можно проделывать с обычными объектами:

```
var obj = {};              // Создать простой объект
obj[1] = "one";            // Индексировать его целыми числами
```

Особенность массивов состоит в том, что при использовании имен свойств, которые являются неотрицательными целыми числами, массивы автоматически определяют значение свойства length. Например, выше был создан массив arr с единственным элементом. Затем были присвоены значения его элементам с индексами 1, 2 и 3. В результате этих операций значение свойства length массива изменилось и стало равным 4.

Следует четко отличать индексы в массиве от имен свойств объектов. Все индексы являются именами свойств, но только свойства с именами, представленными целыми числами являются индексами. Все массивы являются объектами, и вы можете добавлять к ним свойства с любыми именами. Однако если вы затрагиваете свойства, которые являются индексами массива, массивы реагируют на это, обновляя значение свойства length при необходимости.

Обратите внимание, что в качестве индексов массивов допускается использовать отрицательные и не целые числа. В этом случае числа преобразуются в строки, которые используются как имена свойств.

Добавление и удаление элементов массивов.

Самый простой способ добавить элементы в массив заключается в том, чтобы присвоить значения новым индексам. Для добавления одного или более элементов в конец массива можно также использовать метод push():

```
var arr = [];              // Создать пустой массив
arr.push('zero');          // Добавить значение в конец
arr.push('one',2);         // Добавить еще два значения
```

Добавить элемент в конец массива можно также, присвоив значение элементу arr[arr.length]. Для вставки элемента в начало массива можно использовать метод unshift(), при этом существующие элементы в массиве смещаются в позиции с более высокими индексами.

Удалять элементы массива можно с помощью оператора delete, как обычные свойства объектов:

```
var arr = [1,2,'three'];
delete arr[2];
2 in arr;                  // false, индекс 2 в массиве не определен
arr.length;                // 3: оператор delete не изменяет свойство length массива
```

Удаление элемента напоминает (но несколько отличается) присваивание значения undefined этому элементу. Обратите внимание, что применение оператора delete к элементу массива не изменяет значение свойства length и не сдвигает вниз элементы с более высокими индексами, чтобы заполнить пустоту, оставшуюся после удаления элемента.

Кроме того, имеется возможность удалять элементы в конце массива простым присваиванием нового значения свойству length. Массивы имеют метод pop() (противоположный методу push()), который уменьшает длину массива на 1 и возвращает значение удаленного элемента. Также имеется метод shift() (противоположный методу unshift()), который удаляет элемент в начале массива. В отличие от оператора delete, метод shift() сдвигает все элементы вниз на позицию ниже их текущих индексов.

Наконец существует многоцелевой метод splice(), позволяющий вставлять, удалять и замещать элементы массивов. Он изменяет значение свойства length и сдвигает элементы массива с более низкими или высокими индексами по мере необходимости. Все эти методы мы разберем чуть позже.

Многомерные массивы.

JavaScript не поддерживает «настоящие» многомерные массивы, но позволяет неплохо имитировать их при помощи массивов из массивов. Для доступа к элементу данных в массиве массивов достаточно дважды использовать оператор [].

Например, предположим, что переменная matrix - это массив массивов чисел. Каждый элемент matrix[x] - это массив чисел. Для доступа к определенному числу в массиве можно использовать выражение matrix[x][y]. Ниже приводится конкретный пример, где двумерный массив используется в качестве таблицы умножения:

```
// Создать многомерный массив
var table = new Array(10);                    // В таблице 10 строк

for(var i = 0; i < table.length; i++)
        table[i] = new Array(10);             // В каждой строке 10 столбцов

// Инициализировать массив и вывести на консоль
for(var row = 0, str = ''; row < table.length; row++) {
        for(var col = 0; col < table[row].length; col++) {
                table[row][col] = (row+1)*(col+1);
                str += table[row][col] + '  ';
        }
        console.log(str + '\n');
        str = '';
}
```

Методы класса Array.

Стандарт ECMAScript определяет в составе Array.prototype множество удобных функций для работы с массивами, которые доступны как методы любого массива. Эти методы будут представлены в следующих подразделах.

Метод join().

Метод Array.join() преобразует все элементы массива в строки, объединяет их и возвращает получившуюся строку. В необязательном аргументе методу можно передать строку, которая будет использоваться для отделения элементов в строке результата. Если строка-разделитель не указана, используется запятая. Например, следующий фрагмент дает в результате строку "1,2,3":

```
var arr = [1,2,3];

arr.join();                    // '1,2,3'
arr.join("-");          // '1-2-3'
```

Метод reverse().

Метод Array.reverse() меняет порядок следования элементов в массиве на обратный и возвращает переупорядоченный массив. Перестановка выполняется непосредственно в исходном массиве, т.е. этот метод не создает новый массив с переупорядоченными элементами, а переупорядочивает их в уже существующем массиве. Например, следующий фрагмент, где используются методы reverse() и join(), дает в результате строку "3,2,1":

```
var arr = [1,2,3];
arr.reverse().join();          // "3,2,1"
```

Метод sort()

Метод Array.sort() сортирует элементы в исходном массиве и возвращает отсортированный массив. Если метод sort() вызывается без аргументов, сортировка выполняется в алфавитном порядке (для сравнения элементы временно преобразуются в строки, если это необходимо). Неопределенные элементы переносятся в конец массива.

Для сортировки в каком-либо ином порядке, отличном от алфавитного, методу sort() можно передать функцию сравнения в качестве аргумента. Эта функция устанавливает, какой из двух ее аргументов должен следовать раньше в отсортированном списке. Если первый аргумент должен предшествовать второму, функция сравнения должна возвращать отрицательное число. Если первый аргумент должен следовать за вторым в отсортированном массиве, то функция должна возвращать число больше нуля. А если два значения эквивалентны (т.е. порядок их следования не важен), функция сравнения должна возвращать 0:

```
var arr = [33, 4, 1111, 222];
arr.sort();                    // Алфавитный порядок: 1111, 222, 33, 4

arr.sort(function(a,b) {
        // Числовой порядок: 4, 33, 222, 1111
        return a-b;            // Возвращает значение < 0, 0 или > 0
                               // в зависимости от порядка сортировки a и b
});

// Сортируем в обратном направлении, от большего к меньшему
```

```
arr.sort(function(a,b) {return b-a});
```

Обратите внимание, насколько удобно использовать в этом фрагменте неименованную функцию. Функция сравнения используется только здесь, поэтому нет необходимости давать ей имя.

Метод concat()

Метод Array.concat() создает и возвращает новый массив, содержащий элементы исходного массива, для которого был вызван метод concat(), и значения всех аргументов, переданных методу concat(). Если какой-либо из этих аргументов сам является массивом, его элементы добавляются в возвращаемый массив. Следует, однако, отметить, что рекурсивного превращения массива из массивов в одномерный массив не происходит. Метод concat() не изменяет исходный массив. Ниже приводится несколько примеров:

```
var arr = [1,2,3];
arr.concat(4, 5);                    // Вернет          [1,2,3,4,5]
arr.concat([4,5]);                   // Вернет          [1,2,3,4,5]
arr.concat([4,5],[6,7])    // Вернет          [1,2,3,4,5,6,7]
arr.concat(4, [5,[6,7]]) // Вернет   [1,2,3,4,5,[6,7]]
```

Метод slice()

Метод Array.slice() возвращает фрагмент, или подмассив, указанного массива. Два аргумента метода определяют начало и конец возвращаемого фрагмента. Возвращаемый массив содержит элемент, номер которого указан в первом аргументе, плюс все последующие элементы, вплоть до (но не включая) элемента, номер которого указан во втором аргументе.

Если указан только один аргумент, возвращаемый массив содержит все элементы от начальной позиции до конца массива. Если какой-либо из аргументов имеет отрицательное значение, он определяет номер элемента относительно конца массива. Так, аргументу -1 соответствует последний элемент массива, а аргументу -3 - третий элемент массива с конца. Вот несколько примеров:

```
var arr = [1,2,3,4,5];
arr.slice(0,3);              // Вернет [1,2,3]
arr.slice(3);                // Вернет [4,5]
arr.slice(1,-1);  // Вернет [2,3,4]
arr.slice(-3,-2); // Вернет [3]
```

Метод splice()

Метод Array.splice() - это универсальный метод, выполняющий вставку или удаление элементов массива. В отличие от методов slice() и concat(), метод splice() изменяет исходный массив, относительно которого он был вызван. Обратите внимание, что методы splice() и slice() имеют очень похожие имена, но выполняют совершенно разные операции.

Метод splice() может удалять элементы из массива, вставлять новые элементы или выполнять обе операции одновременно. Элементы массива при необходимости смещаются, чтобы после вставки или удаления образовывалась непрерывная последовательность.

Первый аргумент метода splice() определяет позицию в массиве, начиная с которой будет выполняться вставка и/или удаление. Второй аргумент определяет количество элементов, которые должны быть удалены (вырезаны) из массива. Если второй аргумент опущен, удаляются все элементы массива от указанного до конца массива. Метод splice() возвращает массив удаленных элементов или (если ни один из элементов не был удален) пустой массив.

Первые два аргумента метода splice() определяют элементы массива, подлежащие удалению. За этими аргументами может следовать любое количество дополнительных аргументов, определяющих элементы, которые будут вставлены в массив, начиная с позиции, указанной в первом аргументе.

```
var arr = [1,2,3,4,5,6,7,8];
arr.splice(4);                                  // Вернет [5,6,7,8], arr = [1,2,3,4]
arr.splice(1,2);                          // Вернет [2,3], arr = [1,4]
arr.splice(1,1);                          // Вернет [4]; arr = [1]

arr = [1,2,3,4,5];
arr.splice(2,0,'a','b');              // Вернет []; arr = [1,2,'a','b',3,4,5]
```

Методы push() и pop()

Методы push() и pop() позволяют работать с массивами как со стеками. Метод push() добавляет один или несколько новых элементов в конец массива и возвращает его новую длину. Метод pop() выполняет обратную операцию - удаляет последний элемент массива, уменьшает длину массива и возвращает удаленное им значение. Обратите внимание, что оба эти метода изменяют исходный массив, а не создают его модифицированную копию.

Методы unshift() и shift().

Методы unshift() и shift() ведут себя почти так же, как push() и pop(), за исключением того, что они вставляют и удаляют элементы в начале массива, а не в конце. Метод unshift() смещает существующие элементы в сторону больших индексов для освобождения места, добавляет элемент или элементы в начало массива и возвращает новую длину массива. Метод shift() удаляет и возвращает первый элемент массива, смещая все последующие элементы на одну позицию вниз, чтобы занять место, освободившееся в начале массива.

ЛИТЕРАТУРА И ДРУГИЕ ИСТОЧНИКИ

Самый лучший общедоступный ресурс с точки зрения простоты изложения материала – это «Современный учебник по Java Script», доступный по ссылке https://learn.javascript.ru/books .

Еще один учебник Антон Кармазин «Выразительный JavaScript», доступен по ссылке https://www.gitbook.com/book/karmazzin/eloquentjavascript_ru/details .

Руководство, которое можно использовать в качестве карманного справочника - http://bonsaiden.github.io/JavaScript-Garden/ru/ .

Данных ресурсов достаточно, чтобы получить начальные навыки владения языком программирования.

Современное печатное издание, которое можно порекомендовать для новичков – Закас Н. JavaScript для профессиональных веб-разработчиков / (Пер. с англ. А. Лютича). – СПб.: Питер, 2015. – 960с.

ВОПРОСЫ ДЛЯ САМОПРОВЕРКИ

1. Дайте определение понятию «объект».
2. Что определяют атрибуты свойства объекта?
3. Как создать объект при помощи инициализатора? Приведите пример.
4. Использование функции-конструктора для создания объекта. Приведите пример.
5. Как можно создать объект при помощи Object.create?
6. Как можно задать свойство объекта?
7. Варианты перечисления свойств объектов?
8. Как можно удалить свойство объекта?
9. Каким образом можно проверить наличие свойства объекта?
10. Как можно задать метод объекта?
11. Дайте определение геттеров и сеттеров. Приведите примеры.
12. Как можно сравнить объекты?
13. Что такое прототип и как осуществляется наследование?
14. Что такое сериализации объекта?
15. Дайте определение массива. Как можно создать массив?
16. Как можно изменять элементы массива?
17. Приведите пример организации многомерного массива.